

Practical-1

Name: Sahil Rahane

PRN:- 202501050009

1.1 Calculate Momentum

Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum is calculated using the formula: where: is the mass of the object (in kilograms). is the velocity of the object (in meters per second).

Input Format:

- A single floating-point number representing the mass of the object in kilograms.
- A single floating-point number representing the velocity of the object in meters per second.

Output Format:

- The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).

Code:

```
mass = float(input()) velocity = float(input())
```

```
momentum = mass*velocity print(f'{momentum:.2f}kgm/s')
```

The screenshot shows the CODETANTRA IDE interface. On the left, a sidebar lists the course 'Essentials of Data Science Laboratory - 2304102L - 2026' and various practical assignments. The main editor area displays the '1.1.1. Calculate Momentum' assignment, which includes the problem description, the formula $p = m \times v$, and the input/output formats. On the right, the code editor shows the Python solution code:

```
1 m=float(input())
2 v=float(input())
3 p=m*v
4 print(f'{p:.2f}kgm/s')
```

1.1.2. Conditional Calculation Based on the Number of Digits

Write a Python program that accepts an integer n as input. Depending on the number of digits in n .

Constraints:

$$1 \leq n \leq 999$$

Input Format:

- The input consists of a single integer n .

Output Format:

- If n is a single-digit number, print its square.
- If n is a two-digit number, print its square root (rounded to two decimal places).
- If n is a three-digit number, print its cube root (rounded to two decimal places).
- Else print "Invalid".

Code:

```
n=int(input()) if 0<=n and n<=9:
```

```
print(n*n) elif 10<=n and n<=99:
```

```
print(f"{n**(1/2):.2f}") elif 100<=n and n<=999:
```

```
print(f"{n**(1/3):.2f}") else:
```

```
print("Invalid")
```

The screenshot displays a web-based IDE interface. On the left, a sidebar shows a course structure for 'Essentials of Data Science Laboratory - 2304102L - 2026'. The main area is titled '1.1.2. Conditional Calculation Based on the Number of Digits in n'. It contains the problem description, constraints ($1 \leq n \leq 999$), input format (a single integer n), and output format (square, square root, cube root, or 'Invalid'). Below this is a 'Sample Test Cases' section. On the right, a code editor shows the Python implementation of the program, which uses conditional statements to calculate the square, square root, or cube root based on the number of digits in n . The code is as follows:

```
1 import math
2 n=int(input())
3 if(1<=n <=9):
4     print(n*n)
5 elif(10<=n <=99):
6     print(f"{math.sqrt(n):.2f}")
7 elif(100<=n <=999):
8     print(f"{math.cbrt(n):.2f}")
9 else:
10    print("Invalid")
11
```

At the bottom of the IDE, there are buttons for 'Terminal', 'Test cases', 'Submit', 'Reset', and 'Next'.

1.1.3. Days between Two Dates

Write a Python program to calculate number of days between two dates.

Input Format:

- The first line contains the first date in the format YYYY-MM-DD
- The second line contains the second date in the format YYYY-MM-DD
- An integer representing the number of days between the given dates.

Code: `from datetime import datetime date1=input().strip() date2=input().strip()`

`d1=datetime.strptime(date1,"%Y-%m-%d") d2=datetime.strptime(date2,"%Y-%m-%d") difference = d2-d1`
`print(difference.days)`

The screenshot shows a web-based Python IDE interface. On the left, a sidebar lists course content under 'Essentials of Data Science Laboratory - 2304102L - 2026'. The main area displays a lab assignment titled '1.1.3. Days between Two Dates'. It includes instructions to write a Python program to calculate the number of days between two dates, with input and output formats specified. The input format requires two dates in YYYY-MM-DD format. The output format requires an integer representing the number of days. A note states that the first date should be earlier than the second. Below the instructions is a 'Sample Test Cases' section. On the right, a code editor shows the Python code for the solution, which uses the datetime module to parse the input dates and calculate the difference in days. The code is as follows:

```
1 from datetime import date
2
3 from datetime import datetime
4
5 date1 = input().strip()
6 date2 = input().strip()
7
8 d1 = datetime.strptime(date1, "%Y-%m-%d")
9 d2 = datetime.strptime(date2, "%Y-%m-%d")
10
11 difference = d2 - d1
12
13 print(difference.days)
```

1.1.4. Reverse a Number

Write a Python program to reverse the digits of the number and print the reversed number.

Input Format:

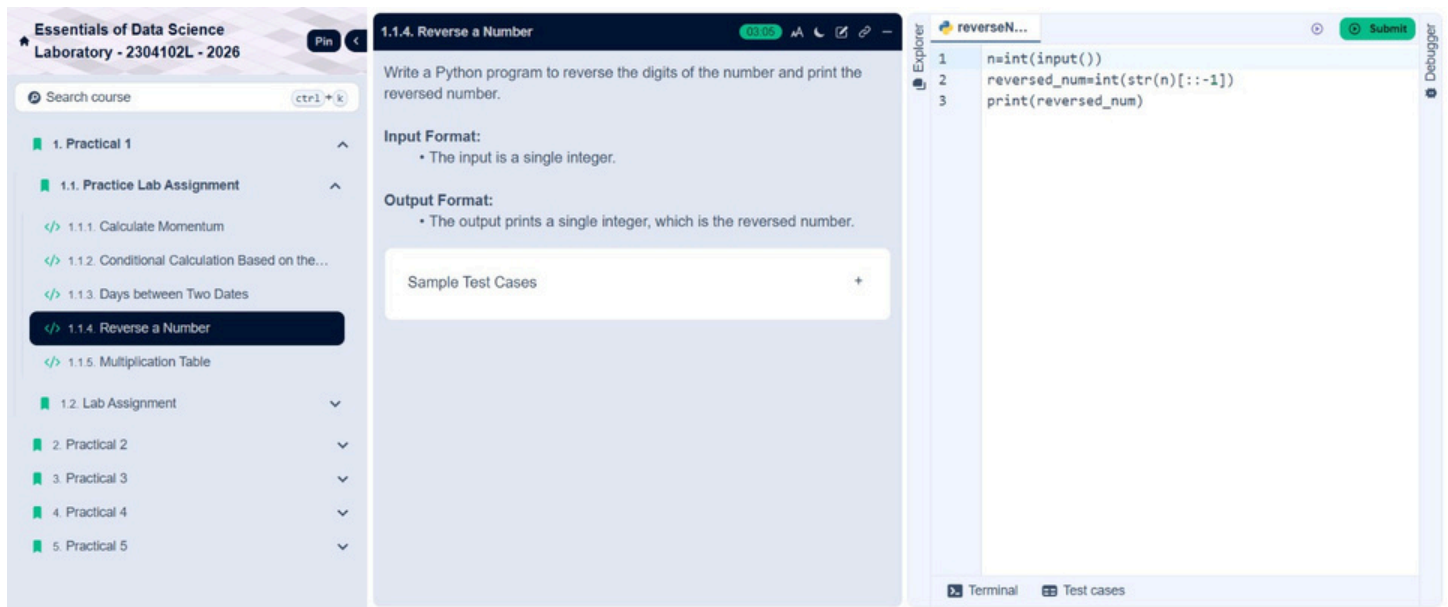
- The input is a single integer.

Output Format:

- The output prints a single integer, which is the reversed number.

Code:

`num = int(input()) reversed_num= int(str(num)[::-1]) print(reversed_num)`



1.1.5. Multiplication Table

Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

Input Format:

- The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

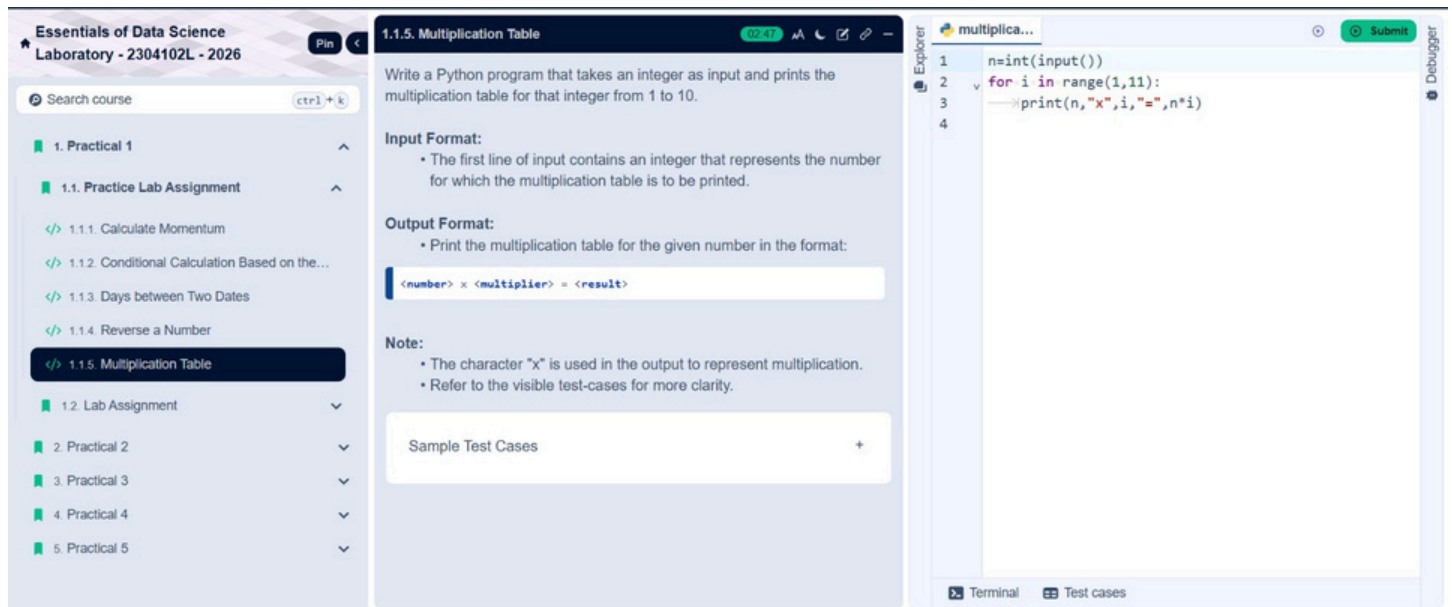
Output Format:

- Print the multiplication table for the given number in the format: **<number> x <multiplier> = <result>**

Code:

```
num = int(input()) for i in range(1,11):
```

```
print(f"{num} x {i} = {num * i}")
```



1.2.1. Pass or Fail

Write a Python program that accepts the number of courses and the marks of a student in those courses.

The grade is determined based on the aggregate percentage:

- If the aggregate percentage is greater than 75, the grade is Distinction.
- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

- The first input will be an integer , the number of courses.
- The second input will be integers representing the marks of the student in each of the courses, separated by a space.

Output Format:

If the student passes all courses:

- Print the aggregate percentage (formatted to two decimal places).
- Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

- "Fail".

Code:

```
def cal(marks, total_courses):
```

```
    if any(mark < 40 for mark in marks):
```

```
        return "Fail"
```

```
    total_marks = sum(marks) aggregate_percentage=(total_marks/(total_courses *100))*100 if
```

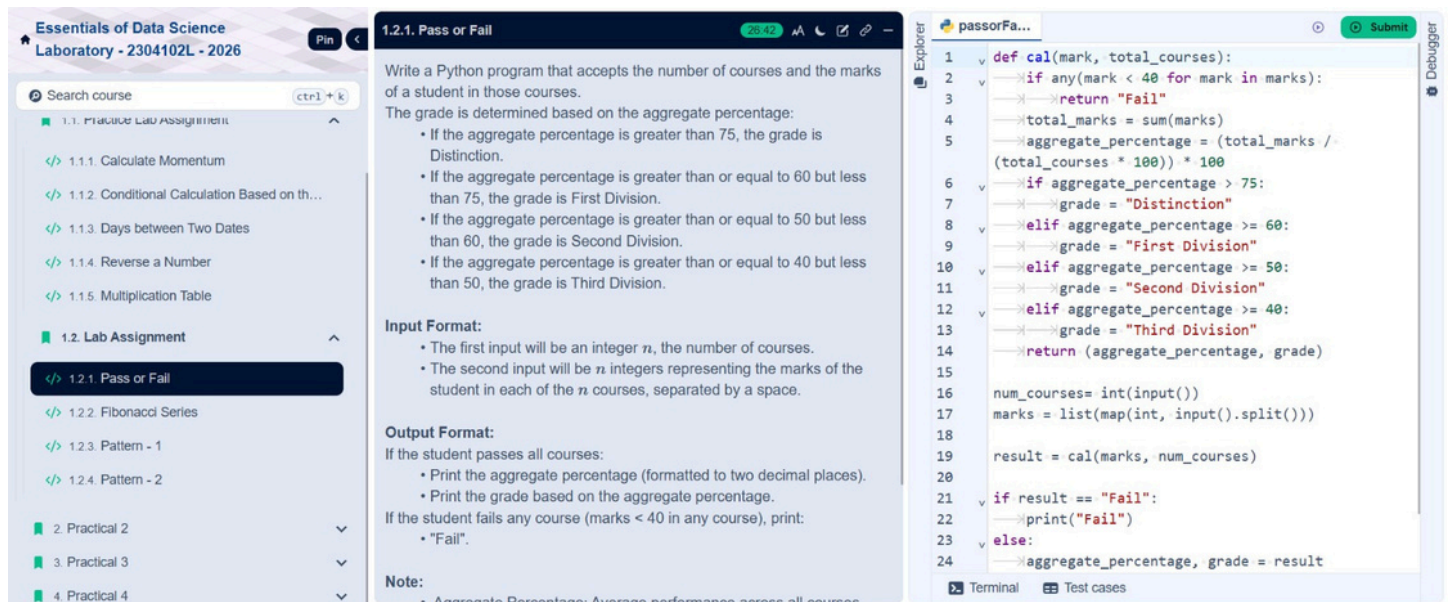
```
    aggregate_percentage >75: grade="Distinction" elif aggregate_percentage >=60: grade= "First Division"
```

```
    elif aggregate_percentage >=50: grade= "Second Division" elif aggregate_percentage >=40: grade= "Third Division"
```

```
return (aggregate_percentage, grade) num_courses= int(input()) marks= list(map(int, input().split())) result=
cal(marks, num_courses) if result == "Fail":
```

```
print("Fail") else:
```

```
aggregate_percentage, grade=result print(f'Aggregate Percentage: {aggregate_percentage:.2f}') print(f'Grade:
{grade}')
```



1.2.2. Fibonacci Series

Write a Python program that uses recursion to print the first terms of the Fibonacci series.

Input Format:

- A single integer representing the number of terms to generate.

Output Format:

- A single line containing the first terms of the Fibonacci sequence, separated by spaces.

Code: def fibonacci(n): if n == 1:

```
return 0 elif n == 2:
```

```
return 1 else:
```

```
return fibonacci(n - 1) + fibonacci(n - 2) n = int(input()) for i in range(1, n + 1):
```

```
print(fibonacci(i), end=" ")
```

Essentials of Data Science Laboratory - 2304102L - 2026

Search course ctrl+k

- 1.1.1. Calculate Momentum
- 1.1.2. Conditional Calculation Based on th...
- 1.1.3. Days between Two Dates
- 1.1.4. Reverse a Number
- 1.1.5. Multiplication Table
- 1.2. Lab Assignment**
 - 1.2.1. Pass or Fail
 - 1.2.2. Fibonacci Series**
 - 1.2.3. Pattern - 1
 - 1.2.4. Pattern - 2
- 2. Practical 2
- 3. Practical 3
- 4. Practical 4

1.2.2. Fibonacci Series

Write a Python program that uses recursion to print the first n terms of the Fibonacci series.

Input Format:

- A single integer n representing the number of terms to generate.

Output Format:

- A single line containing the first n terms of the Fibonacci sequence, separated by spaces.

Sample Test Cases

```

1 #def fibonacci(n):
2
3 def fibonacci(n,a=0,b=1):
4     if n==1:
5         return a
6     else:
7
8         return fibonacci(n-1,b,a+b)
9
10
11 n = int(input())
12 for i in range(1, n + 1):
13     print(fibonacci(i), end=" ")
14

```

1.2.3. Pattern -1

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

- The input is an integer, representing the number of rows in the pattern. **Output Format**
- The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Code:

```
n = int(input())
for i in range (1, n+1):
    print('* ' * i)
```

Essentials of Data Science Laboratory - 2304102L - 2026

Search course ctrl+k

- 1.1.1. Calculate Momentum
- 1.1.2. Conditional Calculation Based on th...
- 1.1.3. Days between Two Dates
- 1.1.4. Reverse a Number
- 1.1.5. Multiplication Table
- 1.2. Lab Assignment**
 - 1.2.1. Pass or Fail
 - 1.2.2. Fibonacci Series
 - 1.2.3. Pattern - 1**
 - 1.2.4. Pattern - 2
- 2. Practical 2
- 3. Practical 3
- 4. Practical 4

1.2.3. Pattern - 1

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format

- The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Note: Refer to the displayed test cases for the sample pattern.

Sample Test Cases

```

1 n = int(input())
2 for i in range(1, n + 1):
3     print("* " * i)

```

1.2.4. Pattern -2

Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format:

- The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Code:

```
n = int(input()) for i in range(1, n+1): for j in range(1, i+1):
```

```
print(j,end=" ") print()
```

The screenshot displays a web application interface for a course titled "Essentials of Data Science Laboratory - 2304102L - 2026". On the left, a sidebar lists various lab assignments, with "1.2.4. Pattern - 2" selected. The main content area shows the problem statement: "Write a Python program to print a right-angled triangle pattern of numbers." It includes the input and output formats, and a note to refer to displayed test cases. Below this is a "Sample Test Cases" section. On the right, a code editor shows the Python solution:

```
n = int(input())
for i in range(1, n + 1):
    for j in range(1, i + 1):
        print(j, end=" ")
    print()
```